Computational Neuroscience — Week 4

Information and Coding Principles

Entropy, Noise, and How Much a Spike Train Tells You

These notes cover lectures 1–3 — defining information and entropy, building up to mutual information, computing it from real spike trains, and finally asking what it says about why neural codes are shaped the way they are. Green boxes give the plain-words version of each idea before the mathematics; rust boxes flag the places where the obvious reading is the wrong one.

Contents

I Entropy and Information **2**

1 The Question **2**

2 Surprise **3**

3 Why a Logarithm? **3**

4 Entropy: Average Information **3**

5 Entropy as Coding Capacity **4**

6 Bringing In the Stimulus **5**

7 Noise Entropy **5**

8 Mutual Information **6**

9 The Two Limits **6**

10 Continuous Variables **6**

11 Information Measures Dependence **7**

12 The Recipe **7**

13 Where This Leaves Us **8**

II Calculating Information in Spike Trains **8**

14 Two Methods **8**

15 Method 1: Information in Spike Patterns **9**

16 The LGN Worked Example **10**

17 Sampling and Bias **10**

18 Method 2: Information in Single Spikes **11**

19 What the Formula Says 12

20 Place Fields 12

21 Where This Leaves Us 13

III Information and Coding Efficiency 13

22 The Challenge of Natural Stimuli 13

23 What Makes a Good Code 14

24 Laughlin's Test 15

25 Adapting in Real Time 15

26 Feature Adaptation 16

27 Redundancy Reduction 16

28 Sparse Coding 17

29 Where This Leaves Us 17

Part I

Entropy and Information

Lecture 1 — surprise, entropy, noise entropy, and mutual information.

1. The Question

Weeks 2 and 3 built models: how a stimulus produces spikes, and how to read the stimulus back out. This week asks a different kind of question about the same spike trains.

How good is my code?

Treat a spike train as a string of ones and zeros. How much can such a string possibly say — and how much of what it says is actually about the stimulus?

[Lecture slide omitted from the public copy.] The setup, reusing the random-dot task from Week 3. A spike train is written as a row of bins, each holding a spike or a silence.

The idea in plain words

Notice this is not a question about *accuracy*. A decoder can be judged by how often it gets the stimulus right. Information theory instead asks about **capacity** and **efficiency** — how much a channel could possibly carry, and how much of that capacity is being used. It is a way of grading the code itself rather than any particular attempt to read it.

2. Surprise

Look at a long run of empty bins and then a spike. You feel a small jolt. That jolt is the thing being quantified.

Let p be the probability of a spike in a given time bin, so $1-p$ is the probability of silence. Then

$$\text{information}(1) = -\log_2 p, \qquad \text{information}(0) = -\log_2(1-p).$$

[Lecture slide omitted from the public copy.] Information as surprise. A rare event carries more information than a common one.

The idea in plain words

The minus sign is there because probabilities are less than one and their logarithms are negative; it just makes information positive. The shape of the definition says something simple: **the less likely an event was, the more you learn when it happens**.
An event that was certain ($p = 1$) carries $-\log_2 1 = 0$ bits — learning it happened tells you nothing, because you already knew. An event with $p = 1/2$ carries exactly one bit.

3. Why a Logarithm?

The logarithm is not decoration. It is what makes information *add up*.

[Lecture slide omitted from the public copy.] The squash court argument. Each bit of information halves the region the ball could be in.

Suppose your opponent can place the ball anywhere in the court with equal probability. Tell them which half it is in — that is one bit. Tell them which half of that half — another bit. Two bits pins the ball to one quarter.

In probabilities you *multiply*: the probability of being in the left half *and* the front half is $\frac{1}{2}\times\frac{1}{2} = \frac{1}{4}$. Taking $-\log_2$ turns that product into a sum: $1 + 1 = 2$ bits.

What a bit is

One bit is the information in one yes/no answer about equally likely alternatives. Each additional bit cuts the remaining possibilities by a further factor of two.

4. Entropy: Average Information

Information is defined per event. **Entropy** is its average over all events.

$$H = -\sum_i p_i \log_2 p_i \qquad \text{or, for a continuous variable,} \qquad H = -\int dx\, p(x) \log_2 p(x)$$

[Lecture slide omitted from the public copy.] Two distributions with very different entropies. The peaked one on the left is predictable, so its average surprise is low; the spread-out one on the right is not.

Watch out — entropy is a property of the distribution, not of any one outcome

A single event has information. A *distribution* has entropy. Asking for "the entropy of this spike" is a category error — entropy describes how variable the source is, averaged over everything it might have done.

The idea in plain words

The most useful reading: entropy counts the **number of yes/no questions** you need to pin down the variable. That is exactly what the squash court was doing, and it is why the units are bits.

Logs are always base 2 here, and from now on the base is dropped and assumed.

A worked example

You park in one of eight spots and the attendant will only answer yes or no. How many questions to find the car?

[Lecture slide omitted from the public copy.] Eight equally likely spots, and the entropy worked out on the slide.

Each spot has $p_i = 1/8$, so

$$H = -\sum_{i=1}^{8} \tfrac{1}{8} \log_2 \tfrac{1}{8} = -\sum_{i=1}^{8} \tfrac{1}{8} \times (-3) = 3 \text{ bits}$$

using $8 = 2^3$, hence $\log_2 \frac{1}{8} = -3$. And three questions — left half? top half? left of that? — is exactly what it takes. The entropy and the number of questions are the same number, which is the point.

5. Entropy as Coding Capacity

Now apply this to spike trains.

[Lecture slide omitted from the public copy.] Three spike trains with very different amounts of variability.

The idea in plain words

Encoding requires the output to *change* when the input changes. A neuron that fires in every bin regardless, or never fires at all, cannot signal anything — there is no room in its output for the stimulus to leave a mark.
So variability is not merely noise to be tolerated. It is the raw material of a code, and entropy measures how much of it there is. **Higher entropy means more capacity to represent.**

Where the maximum is

For a binary code with spike probability p,

$$H = -p \log p - (1-p) \log(1-p)$$

[Lecture slide omitted from the public copy.] The binary entropy function. It peaks at $p = 1/2$, where the two symbols are used equally often, and falls to zero at both extremes.

But the world is not uniform

[Lecture slide omitted from the public copy.] Back to the squash court. A real player cannot place the ball anywhere with equal probability — and that is precisely what makes the game playable.

The idea in plain words

There is a nice double edge here. From the *sender's* point of view, maximum entropy is best: an opponent whose shot distribution is uniform is unpredictable and hard to beat. From the *receiver's* point of view, low entropy is a gift: the more structure there is in $P(x)$, the better you can predict the next shot.
A good player is constantly trying to violate their opponent's model of them. Both halves of this reappear later in the week, when we ask whether neural codes are structured to match the statistics of the natural world.

6. Bringing In the Stimulus

Entropy measures how variable the response is. It says nothing about *why* it varies. Variability driven by the stimulus is signal; variability that is not is noise, and entropy counts both alike.

[Lecture slide omitted from the public copy.] Stimulus and response side by side. Here each of two stimulus directions maps onto a spike or a silence.

Three cases are worth distinguishing. Response perfectly correlated with the stimulus. Response mostly correlated, with occasional misfires. Response with no visible relationship at all.

Watch out — a code that looks noisy may just be a code you have misread

If response and stimulus appear unrelated, one possibility is a genuinely noisy neuron. Another is that you have picked the wrong stimulus variable — perhaps the cell cares about some temporal pattern rather than the feature you chose. That is exactly the question Week 2 was built to answer, and it is worth keeping in view before concluding that a code is bad.

7. Noise Entropy

To make this quantitative, allow for error. Let q be the probability that the response is wrong:

$$P(r_+ \mid +) = 1 - q, \quad P(r_- \mid +) = q, \qquad P(r_- \mid -) = 1 - q, \quad P(r_+ \mid -) = q.$$

[Lecture slide omitted from the public copy.] The error model, with the two entropies that matter written out beneath it.

Total entropy and noise entropy

Total entropy $H[R]$ — the variability of the responses overall.
Noise entropy $H[R \mid s]$ — the variability that remains when the stimulus is held *fixed*.

The idea in plain words

Present the same stimulus many times over. Any spread you still see in the responses cannot be about the stimulus, because the stimulus never changed. That residual spread is the noise entropy, and it is capacity being spent on something other than the message.

8. Mutual Information

Subtract the wasted capacity from the total:

$$I(S;R) \;=\; H[R] \;-\; \sum_{s} P(s)\, H[R \mid s]$$

total entropy − average noise entropy

[Lecture slide omitted from the public copy.] Mutual information written out in full: the entropy of the responses, minus the entropy of the responses given a fixed stimulus, averaged over stimuli.

The idea in plain words

Read it as a budget. The response has a certain total capacity for variety. Some of that variety is squandered on noise. What is left over — the part of the variability that tracks the stimulus — is the information the code actually carries.

[Lecture slide omitted from the public copy.] Left: entropy against spike probability, peaking at $p = 1/2$. Right: mutual information against error probability q. At $q = 0$ the response is perfect and carries the full bit; by $q = 1/2$ responses are at chance and carry nothing.

9. The Two Limits

[Lecture slide omitted from the public copy.] Two cases that can be settled without computing anything.

The opposite extreme is just as quick. If the stimulus predicts the response perfectly, then for each fixed s there is only one possible response, the noise entropy is zero, and

$$I(S;R) \;=\; H[R].$$

All of the code's capacity is being spent on the message. So mutual information runs from 0, for a response that says nothing, up to $H[R]$, for a response that says everything it possibly could.

10. Continuous Variables

Nothing changes in principle when r is continuous.

[Lecture slide omitted from the public copy.] Each pink curve is $P(r \mid s_i)$ weighted by $P(s_i)$; the blue envelope is the total $P(r)$ obtained by summing them. Top: narrow conditionals, well separated — high information. Bottom: the same total distribution, but each conditional is broad and they overlap heavily — low information.

The idea in plain words

Compare the two panels. The blue envelope is about the same in each, so the total entropy is similar — the response uses the same overall range either way. What differs is how much of that range each individual stimulus explores.
When the conditionals are narrow, knowing s pins r down tightly and the noise entropy is small, so most of the total is information. When they are broad and overlapping, a single stimulus already produces nearly the full range of responses, the noise entropy nearly equals the total, and little is left over.

11. Information Measures Dependence

There is a second, equivalent way to define mutual information which makes its meaning clearer. If R and S were independent, their joint distribution would factorise into $P(R)P(S)$. Mutual information is how far the true joint distribution sits from that:

$$I(S;R) \;=\; D_{KL}\big[\,P(R,S)\;,\;P(R)P(S)\,\big]$$

using the Kullback–Leibler divergence, $D_{KL}(P,Q) = \int dx\; P(x) \log \frac{P(x)}{Q(x)}$.

[Lecture slide omitted from the public copy.] Expanding the KL divergence recovers the earlier definition — and, by symmetry, its mirror image.

The idea in plain words

This says mutual information is literally a **measure of how far from independent two variables are**. If they are independent the joint equals the product of marginals, the divergence is zero, and so is the information — the same conclusion reached from a different direction.

Watch out — information is symmetric — and the name reflects that

Expanding the divergence gives

$$I(S;R) = H[R] - \sum_s P(s)H[R \mid s] = H[S] - \sum_r P(r)H[S \mid r].$$

Nothing distinguishes stimulus from response. The information the response carries about the stimulus equals the information the stimulus carries about the response — which is why it is called *mutual*. It is a property of the relationship, not a directed quantity flowing one way.

12. The Recipe

[Lecture slide omitted from the public copy.] The procedure for measuring mutual information from real data.

1. Take one stimulus s and repeat it many times to obtain $P(R \mid s)$.
2. Compute the variability due to noise: the noise entropy $H[R \mid s]$.
3. Repeat for all s and average: $\sum_s P(s)H[R \mid s]$.

4. Compute $P(R) = \sum_s P(s)P(R \mid s)$ and from it the total entropy $H[R]$.

5. Subtract.

Watch out — step 1 is where the experiment lives

Everything here rests on being able to present the *same* stimulus many times and collect the spread of responses. Without repeats there is no way to separate noise entropy from total entropy, and the whole quantity is unmeasurable. This is why information-theoretic experiments are built around repeated presentations of identical stimuli.

13. Where This Leaves Us

- **Information** is surprise: $-\log_2 p$. The logarithm is what makes independent pieces of evidence add rather than multiply.
- **Entropy** is average information — the number of yes/no questions needed to pin the variable down, and a measure of a code's raw capacity. For a binary code it peaks at $p = 1/2$.
- **Noise entropy** is the variability left when the stimulus is held fixed. It is capacity spent on nothing.
- **Mutual information** is the difference: total entropy minus average noise entropy. It runs from 0 when response and stimulus are independent to $H[R]$ when the response is perfectly determined by the stimulus.
- It is equally the KL divergence between the joint distribution and the product of the marginals — a measure of dependence — and it is symmetric in its two arguments.

*To add from lectures: The next sections compute information in real spike trains by two routes: information in **spike patterns**, and information in **single spikes**.*

Part II

Calculating Information in Spike Trains

Lecture 2 — two ways to get a number out of real data: information in spike patterns, and information in single spikes.

14. Two Methods

Part I defined mutual information and gave a recipe for it. This lecture is about actually computing it from recordings, by two routes.

[Lecture slide omitted from the public copy.] Both methods come out of Bill Bialek's group, which pioneered the use of information theory in neuroscience.

The idea in plain words

The two differ in what they treat as the "response". The first chops the spike train into short *patterns* and asks how informative the pattern is. The second asks a narrower question — how much does one spike tell you? — and gets a strikingly simple formula in return.

15. Method 1: Information in Spike Patterns

So far the response has been a firing rate or a single spike. But a spike train has structure: the *arrangement* of spikes may carry more than their count.

[Lecture slide omitted from the public copy.] The question. Sequences of ones and zeros have patterns in them, and those patterns may be informative.

Turning spike trains into words

Bin the spike train at resolution Δt, writing 1 for a spike and 0 for silence. Then chop the resulting string into **words** of length T — so each word has $L = T/\Delta t$ letters.

[Lecture slide omitted from the public copy.] Words of letter size Δt and length T, and the distribution $P(w)$ built by sliding through the data. Silence is the most common word, then words with a single spike in one of the available positions, then combinations.

Two knobs

Δt — the **letter size**, i.e. how finely spike timing is resolved.
L — the **word length**, i.e. how much temporal context the symbol captures.

With $P(w)$ in hand the total entropy is just the entropy of that distribution,

$$H[w] \;=\; -\sum_i P(w_i)\log_2 P(w_i).$$

The noise entropy

Now repeat the *same* stimulus many times and look at the words that occur at one particular moment. Because of trial-to-trial variability you get a distribution of words rather than a single one — but a much narrower distribution than the overall $P(w)$.

[Lecture slide omitted from the public copy.] Total word distribution above, and the distribution given a fixed stimulus below. The narrowing is the information.

The idea in plain words

That narrowing is the whole quantity. Knowing the stimulus collapses your uncertainty about which word will appear, and mutual information measures exactly how much the distribution shrank.

Averaging over time instead of over stimuli

The recipe says to average the noise entropy over all stimuli, which sounds impossible. The trick is to use one long random stimulus and average over *time* instead:

$$H_{\text{noise}} \;=\; \big\langle\, H\big[P(w \mid s_i)\big] \,\big\rangle_i$$

where i indexes time points in the repeated sequence.

[Lecture slide omitted from the public copy.] The recipe applied. Each time point in the repeated stimulus stands in for one sample of s, so the average over stimuli becomes an average over time.

Watch out — this substitution has a name and a condition

Swapping an average over the stimulus ensemble for an average over time is **ergodicity**. It is valid when different stimulus values are visited over time with the same frequency as their probability — which is why the repeated segment has to be long enough to be representative.

16. The LGN Worked Example

Reinagel and Reid (2000) ran exactly this procedure on the LGN.

[Lecture slide omitted from the public copy.] Top: responses to a unique random stimulus, giving the total distribution. Bottom: responses to frozen *white noise — the same segment of noise replayed over and over — giving a PSTH with sharply time-locked events. Right: zooming in, individual events have a jitter of well under a millisecond.*

The idea in plain words

The jitter is the crux. If a spike can wander by a millisecond and still mean the same thing, then binning finer than that millisecond gains you nothing — you would only be resolving noise. So the data itself suggests there is a natural Δt below which extra temporal precision is wasted.
The information calculation turns that suggestion into a measurement.

The result

[Lecture slide omitted from the public copy.] Left: entropy against $1/L$, with the extrapolation to infinite word length. Right: information rate as a function of letter size Δt (vertical) and word length (horizontal). Warm colours are high information rates.

Two things to read off the map:

- Making the bins finer buys more information — but only down to about $\Delta t \approx 2\,\text{ms}$. Below that, the rate stops climbing.
- The information rate saturates around 80–100 bits per second.

The idea in plain words

And that 2 ms matches the jitter seen in the raw rasters. The code is readable at millisecond resolution and no finer, which is a statement about what the LGN's spike timing actually means — obtained without ever building a model of what the neuron encodes.

17. Sampling and Bias

[Lecture slide omitted from the public copy.] The permanent difficulty with information-theoretic quantities.

Watch out — longer words are always undersampled

The number of possible words grows as 2^L. With a finite recording, long words are seen a handful of times or never, so the estimated $P(w)$ is missing most of its support and the estimated entropy comes out *too low*. That is why the entropy curve bends down at large L (small $1/L$) — an artefact of sampling, not a property of the code.

This is not a quirk of this experiment. Information requires the *full* distribution of responses and of stimuli, and there is essentially never enough data to estimate those reliably except in very simple setups.

18. Method 2: Information in Single Spikes

A different question, due to Brenner and colleagues: by how much does knowing that a particular stimulus occurred reduce the entropy of the response?

[Lecture slide omitted from the public copy.] The same LGN data, now asked a narrower question.

The idea in plain words

The appeal is that this can be answered **without knowing what the neuron encodes**. Mutual information quantifies the relationship between input and output without needing a model of that relationship — which sidesteps the worry raised in Part I, that a code might look noisy only because we picked the wrong stimulus variable.

Setting it up

Without knowing the stimulus, the probability of a spike in a bin is the mean rate times the bin width. Knowing it, the probability is the *time-varying* rate times the bin width:

$$P(r{=}1) = \bar{r}\,\Delta t, \qquad P(r{=}1 \mid s) = r(t)\,\Delta t.$$

[Lecture slide omitted from the public copy.] The two probabilities. $r(t)$ is estimated by repeating the stimulus and averaging, which is what the PSTH in the lower panel is.

The entropy difference

Write $p = \bar{r}\Delta t$ and $p(t) = r(t)\Delta t$. The total entropy is the binary entropy of p; the noise entropy is the binary entropy of $p(t)$, averaged over t:

$$I(r,s) = \underbrace{-p\log p - (1-p)\log(1-p)}_{\text{total}} + \underbrace{\frac{1}{T}\int_0^T dt\,\big[p(t)\log p(t) + (1-p(t))\log(1-p(t))\big]}_{\text{noise}}$$

[Lecture slide omitted from the public copy.] Total minus noise, with the ergodic step spelled out: every time t stands in for a sample of s.

The result

[Lecture slide omitted from the public copy.] Assuming $p \ll 1$, expanding $\log(1-p) \approx -p$, and dividing by $\bar{r}\Delta t$ to get information per spike rather than per bin.

$$I(r,s) = \frac{1}{T}\int_0^T dt\, \frac{r(t)}{\bar{r}} \log_2 \frac{r(t)}{\bar{r}}$$

information per spike

19. What the Formula Says

[Lecture slide omitted from the public copy.] The formula and what limits it.

Everything is expressed through the single ratio $r(t)/\bar{r}$ — how far the instantaneous rate departs from the average.

- **No explicit stimulus dependence.** The stimulus appears nowhere in the formula. No coding or decoding model is needed — only a repeated stimulus that is representative of the ensemble.
- **It applies to any event, not just spikes.** r can be the rate of occurrence of any symbol in the code — a particular interspike interval, a particular pattern — which makes it a general way to ask how informative a given feature of the code is.

What limits it

- **Timing precision.** Poor precision blurs $r(t)$, flattening its peaks and troughs. In the limit where $r(t) \to \bar{r}$ the ratio is 1, its logarithm is 0, and the information vanishes — a response that is not modulated by the stimulus says nothing about it.
- **Mean firing rate.** A low $\bar{r}$ makes the ratio large wherever the neuron does fire, so each spike carries more.

Watch out — information per spike is not information per second

A neuron that fires very rarely can be enormously informative *per spike* and still transmit at a low *rate*, simply because it produces so few spikes. The two quantities pull in opposite directions as $\bar{r}$ changes, and which one matters depends on the question being asked.

20. Place Fields

A concrete illustration. A hippocampal **place cell** fires when the animal is in a particular region of space, so the rat's trajectory plays the role of the stimulus.

[Lecture slide omitted from the public copy.] A small, sharply bounded place field. The rat's path crosses it rarely, and when it does the rate jumps abruptly.

The idea in plain words

The useful picture: high information per spike means a **large reduction in uncertainty about where in stimulus space you are**. A spike from the small, sharp-edged cell pins

the rat to a tiny patch of the arena. A spike from the broad cell narrows it down hardly at all.

21. Where This Leaves Us

- **Spike patterns.** Bin at Δt, chop into words of length L, build $P(w)$, and take total entropy minus average noise entropy. Repeats of a frozen stimulus give the noise; ergodicity turns the average over stimuli into an average over time.
- In the LGN this gives 80–100 bits per second, saturating below $\Delta t \approx 2\,\text{ms}$ — matching the observed spike-time jitter.
- **Sampling is the permanent obstacle.** Long words are undersampled and their entropy is biased low; extrapolating in $1/L$ is the standard workaround.
- **Single spikes.** Information per spike is $\frac{1}{T}\int dt\, \frac{r(t)}{\bar{r}} \log_2 \frac{r(t)}{\bar{r}}$, which needs no model of what the neuron encodes and applies to any event in the code.
- Sharply modulated rates and low mean rates both raise information per spike — though not necessarily the information rate.

To add from lectures: Next: information and coding efficiency. What challenges do natural stimuli pose, what does information theory suggest neural systems should do about them, and what principles appear to shape real neural codes?

Part III

Information and Coding Efficiency

Lecture 3 — what natural stimuli demand of a code, and how real codes reshape themselves to meet it.

22. The Challenge of Natural Stimuli

Parts I and II built the tools. This lecture asks what they say about why neural codes look the way they do.

[Lecture slide omitted from the public copy.] A room with a bright view outside, exposed for the interior.

[Lecture slide omitted from the public copy.] The same scene exposed for the view. The camera has to choose; your eye does not.

The idea in plain words

Sitting in that room you would see both the furniture and the landscape at once, in full detail. A camera cannot — it has to be stopped down for one or opened up for the other. That gap between what the eye manages and what a fixed instrument manages is the problem this lecture is about.

Two properties of natural input make it hard:

*[Lecture slide omitted from the public copy.] First, an enormous **dynamic range** — light levels and contrasts varying over orders of magnitude. Second, power-law scaling: the power spectrum of natural images falls off roughly as ω^{-2}.*

[Lecture slide omitted from the public copy.] The power law reflects an absence of any characteristic scale — similar structure appears at many sizes, because of depth and perspective. The same problem appears in sound: fast fluctuations carrying pitch and nuance, riding on a slow envelope of much larger amplitude.

The problem

How can a system with a *limited* range of responses — a firing rate between zero and some maximum — convey fine detail, when its inputs vary over orders of magnitude?

23. What Makes a Good Code

[Lecture slide omitted from the public copy.] Back to the two quantities from Part I. Entropy is maximal when symbols are used equally often; mutual information is that total minus the noise.

The idea in plain words

Of the two terms in $I = H[R] - \langle H[R \mid s] \rangle$, the noise term is largely out of the coder's hands — it is set by the biophysics. The *total* response entropy is not. So the design question becomes: given a fixed set of symbols, how do we use them so that entropy is as large as possible?
And the answer from Part I was: use every symbol equally often.

Histogram equalisation

Suppose the stimulus $s(t)$ has distribution $P(s)$, and we must map it onto a limited range of firing rates — say 0 to 20 Hz, so twenty usable symbols.

To use each symbol equally often, walk along the stimulus distribution and give each symbol an *equal share of the probability mass*: one twentieth of the area under $P(s)$ per symbol. Where the stimulus is common, symbols are packed close together; where it is rare, they are spread thin.

$$r = g(s) = \frac{1}{\alpha} \int^{s} ds' \, P(s')$$

the input/output curve is the *cumulative integral* of the stimulus distribution

[Lecture slide omitted from the public copy.] Efficient coding, after Barlow and Attneave. Match the outputs to the statistical distribution of the inputs, and mutual information between output and input is maximised.

Histogram equalisation

A good encoder's input/output function should be determined by the distribution of its natural inputs.

24. Laughlin's Test

A direct experimental check, from the early 1980s. Laughlin measured the distribution of contrasts a fly would actually encounter in the natural world, then compared the predicted response curve — the cumulative integral of that distribution — against the measured response of the fly's large monopolar cells, which pool signals from the photoreceptors.

[Lecture slide omitted from the public copy.] The measured contrast distribution, its cumulative integral, and the neuron's actual response curve. The match is close.

The idea in plain words

This is a genuinely strong result: a coding principle, applied to independently measured environmental statistics, predicts the shape of a neuron's response curve with no free parameters fitted to the neuron itself.

25. Adapting in Real Time

But a fixed curve tuned over evolutionary time cannot be the whole story. Contrast statistics change enormously when you walk indoors, or simply move your eyes across a room.

[Lecture slide omitted from the public copy.] Different patches of one image have very different contrast distributions — one narrow, one broad. Should a neural system optimise over evolutionary time, or locally and continuously?

The fly experiment

The test: take white noise $s(t)$ and multiply it by a slowly varying envelope $\sigma(t)$. Repeat the *same* $\sigma(t)$ on every trial but a different white noise sample, so that spikes can be sorted by which part of the envelope they occurred in, and an input/output curve computed separately for each.

[Lecture slide omitted from the public copy.] Fly H1. Curves measured during different epochs differ widely — broad where the stimulus varies a lot, sharp where it varies little. But plot them against the stimulus divided by σ*, and they collapse onto a single curve.*

[Lecture slide omitted from the public copy.] The same effect in rat barrel cortex, responding to whisker motion at two different stimulus variances. Again, normalising by the standard deviation collapses the two curves onto one.

The idea in plain words

The collapse is the whole result. It says the neuron is not using a fixed curve at all: it rescales its input axis to match the current spread of the stimulus, and does so continuously as that spread changes. The *shape* is fixed; the *width* tracks the input.

Why it has to

[Lecture slide omitted from the public copy.] As the characteristics of $s(t)$ change, changes can occur both in the input/output function and in the encoded feature.

Watch out — a mismatched curve wastes almost all of its range

Hold the curve fixed and widen the stimulus: most inputs now land in the flat regions, producing either zero or a saturated response, and almost nothing in between. Narrow the stimulus instead and everything crowds into the middle of the curve, using a sliver of the available range.
Either way the code throws away most of its capacity. Matching the curve to the stimulus range is what keeps the full dynamic range in use.

26. Feature Adaptation

Adaptation is not confined to the input/output function. The *feature* the neuron extracts can change with the stimulus statistics too.

The optimal retinal filter

If the retina applies a linear filter to its input, what filter shape maximises information transmission? The answer depends on two things: the power spectrum of natural images, and the signal-to-noise ratio.

[Lecture slide omitted from the public copy.] Predicted receptive fields at high and low light levels, after Atick and Redlich. Bright: a centre–surround "Mexican hat". Dim: a broad, purely positive integrating field.

The idea in plain words

The two regimes want opposite things. In good light the signal is reliable, so the useful move is to *differentiate* — subtract the surround, find edges, discard the redundancy that the ω^{-2} spectrum guarantees. In poor light the signal is buried, so the useful move is to *integrate* — average over a wider area to beat the noise down, even at the cost of blurring. And this is observed: as light levels fall, the surround weakens and the centre broadens.

[Lecture slide omitted from the public copy.] Maximally informative dimensions again, from Part III of Week 2 — choose the filter that maximises the KL divergence between the spike-conditional and prior distributions, which is equivalent to maximising the information a spike carries. Run it under different stimulus ensembles and the recovered filter changes.

27. Redundancy Reduction

Barlow's proposal: spikes are metabolically expensive, so a nervous system should encode as efficiently as it can. What does that imply for a *population*?

[Lecture slide omitted from the public copy.] For two neurons, maximising joint entropy implies coding independently — but correlations turn out to have their uses.

The argument is short. Joint entropy obeys

$$H[R_1, R_2] \;\leq\; H[R_1] + H[R_2]$$

with equality exactly when the two are independent. So to maximise the entropy of the population, the joint distribution should factorise into the product of the marginals — neurons should carry non-overlapping information. That is **redundancy reduction**.

Watch out — redundancy is not simply waste

The picture has softened considerably. Several neurons encoding the same thing allows error correction and more robust coding, and correlations can actively help discrimination. Retinal neurons are in fact observed to be redundant — their joint distribution is far from the product of independent ones. Efficiency is one pressure among several, not the only objective.

28. Sparse Coding

Barlow's later proposal shifts the target: rather than being independent, a population should be **sparse** — as few neurons active at any moment as possible.

Write an image as a weighted sum of basis functions ϕ_i, with coefficients a_i and some residual noise:

$$I(\vec{x}) \;=\; \sum_i a_i\,\phi_i(\vec{x}) \;+\; \epsilon(\vec{x})$$

and choose the basis by minimising reconstruction error *plus* a penalty on how many coefficients are needed:

$$E \;=\; \sum_{\vec{x}} \Big[\, I(\vec{x}) - \sum_i a_i \phi_i(\vec{x}) \Big]^2 \;+\; \lambda \sum_i C(a_i)$$

The idea in plain words

The first term says *reconstruct the image faithfully*; the second says *do it with as few active coefficients as you can.* A simple choice for C is $|a_i|$, and λ sets how hard the sparseness is pushed. Without the penalty, any complete basis reconstructs perfectly and the problem has no interesting answer.

Watch out — a Fourier basis reconstructs perfectly and is not sparse

Sines and cosines can represent any image exactly — but because the natural power spectrum is broad, a great many coefficients are needed for a typical scene. Completeness is not the issue; *how many coefficients are active* is.

[Lecture slide omitted from the public copy.] Optimising the basis for natural images yields localized, oriented, bandpass features — much like V1 receptive fields. After Olshausen and Field, and Bell and Sejnowski.

The idea in plain words

The conclusion runs backwards from the usual direction. Rather than measuring V1 and asking what it computes, you assume only that the code should be sparse for natural images, optimise, and V1-like receptive fields fall out. That makes a case that cortical receptive fields have the shape they do *because* of the statistics of the world they evolved to represent.

29. Where This Leaves Us

[Lecture slide omitted from the public copy.] The three principles this lecture gathered.

- **Coding efficiency.** To use its symbols equally often, a neuron's input/output curve should be the cumulative integral of the stimulus distribution — histogram equalisation. Laughlin's fly data match this prediction with nothing fitted.

- **Adaptation to stimulus statistics.** Real neurons do not hold that curve fixed. Rescaling the input axis by the current stimulus standard deviation collapses curves measured under different conditions onto one, in fly H1 and in barrel cortex alike. The encoded *feature* adapts too — retinal filters shift from differentiating to integrating as the light fails.

- **Sparseness.** Optimising a basis to represent natural scenes with few active coefficients produces localized oriented filters resembling V1.

- Redundancy reduction is a weaker principle than it first appears: correlations buy robustness and can aid discrimination, and real populations are measurably redundant.

[Lecture slide omitted from the public copy.] And what four weeks of coding has not touched. The course has worked on the step from environment to neural activity. How that activity is synthesized into decisions, how decisions become motor commands, and how actions change what is sensed next, all remain open.

> *To add from lectures: Next week moves from data analysis to modelling: the biophysics of coding, neuronal excitability, and simplified models that capture neuronal firing.*